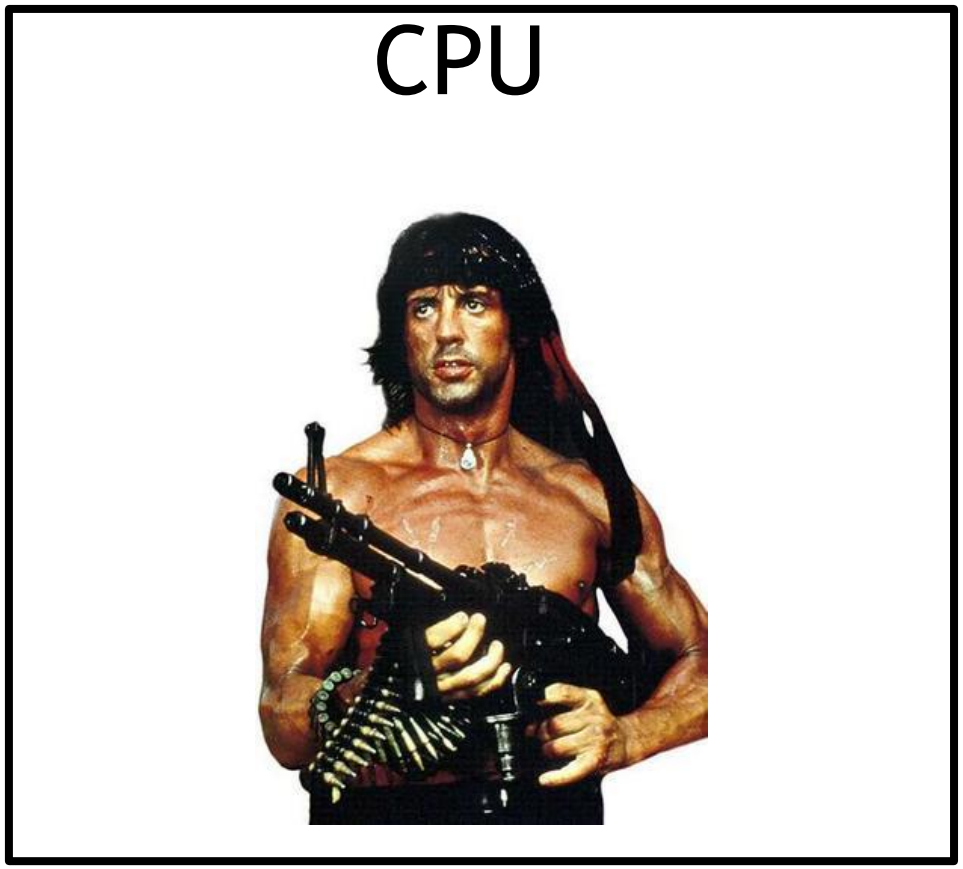
CUDA

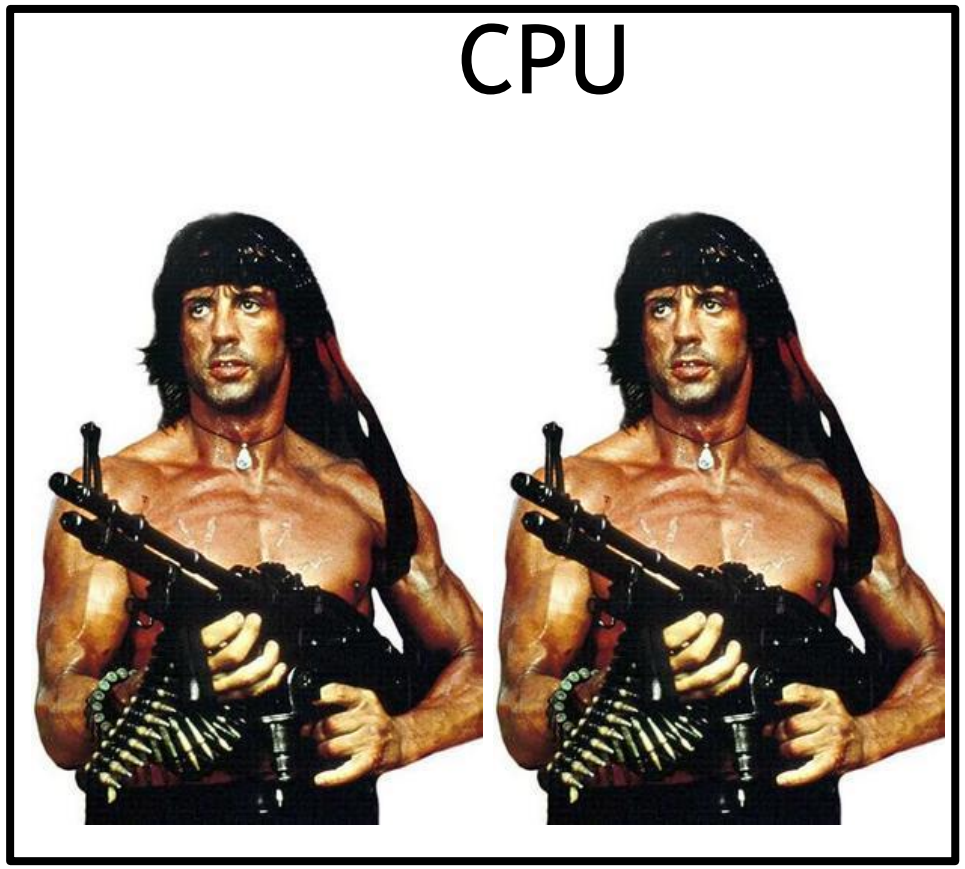
Brad Peterson

The Naïve View

Normal single threaded application



Multithreaded application



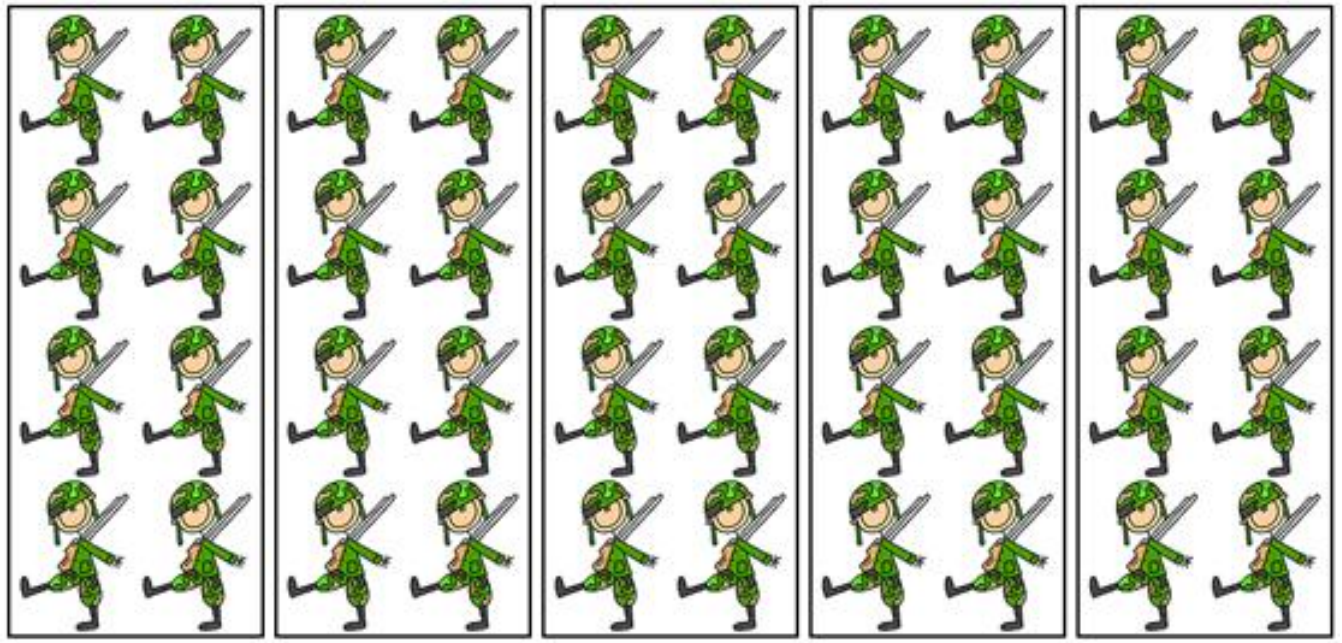
"It's easy. To make a program run twice as fast, just run it on 2 threads!"

Slightly Better Than Naïve

CPU

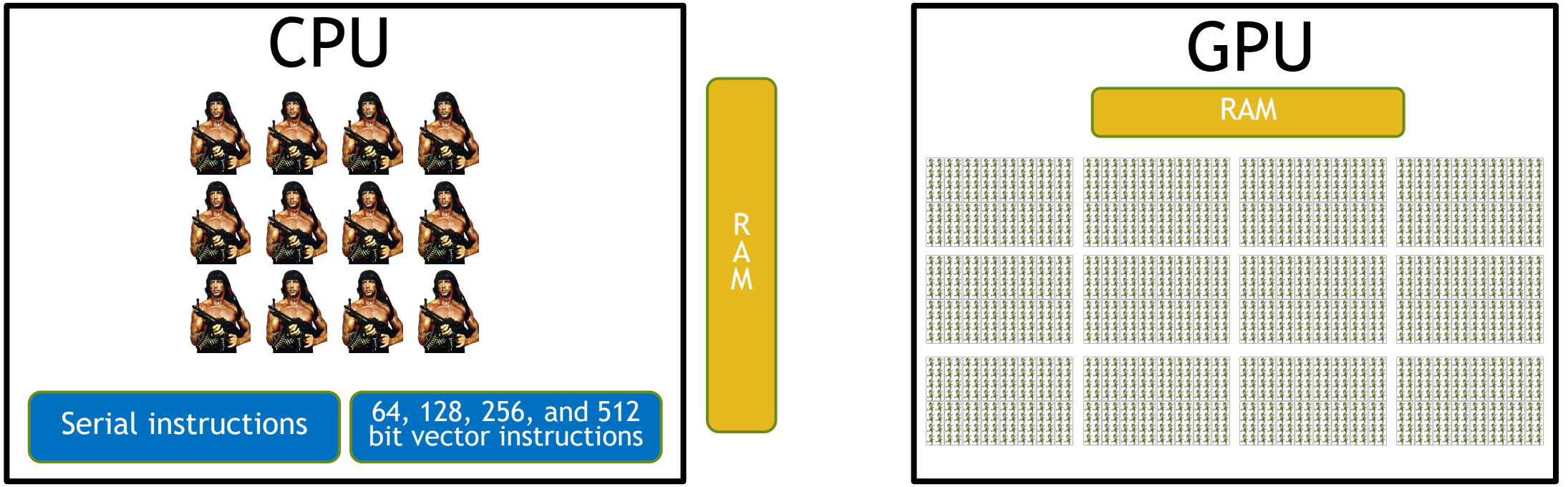
"Just set up a worker model, give each one a job as they need it"

GPU

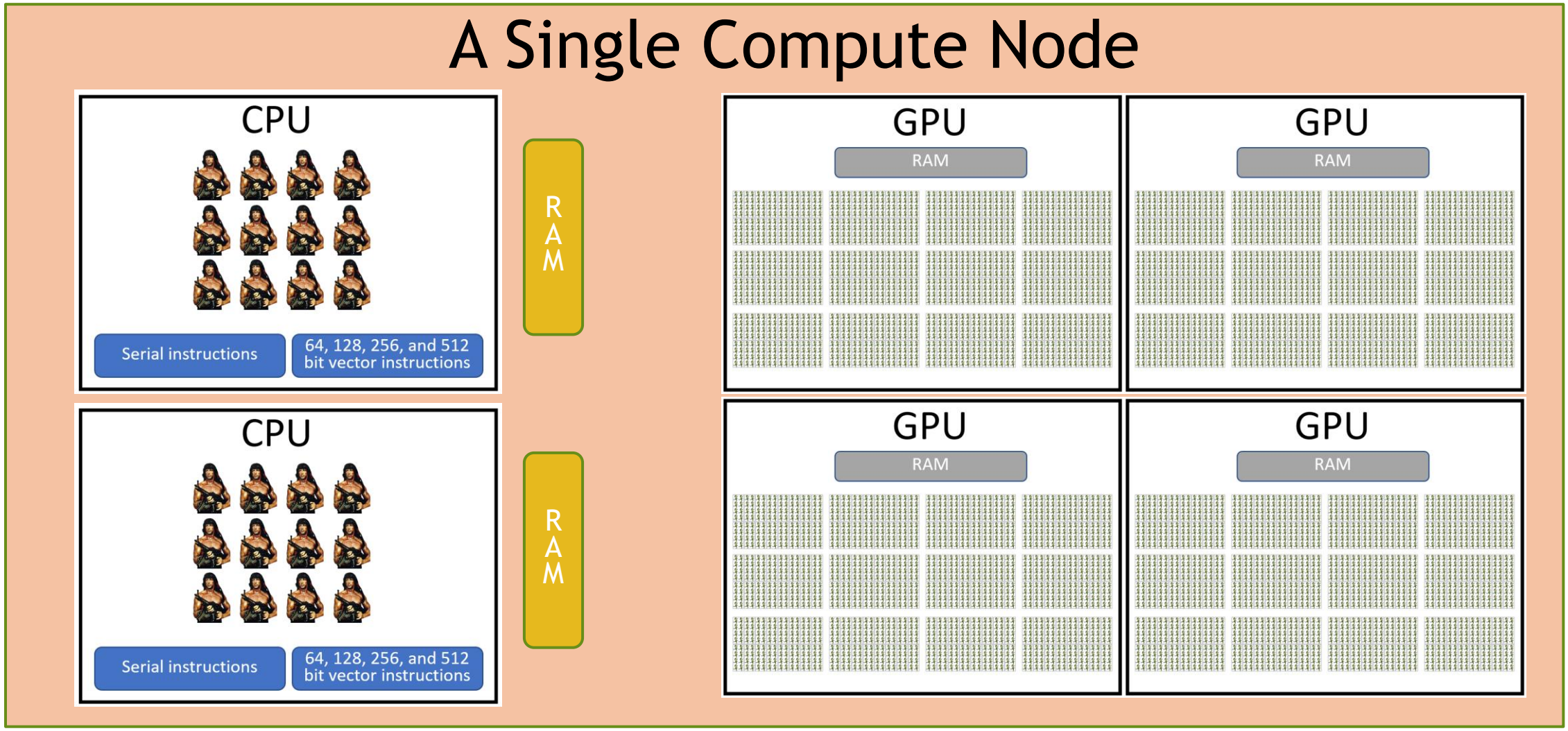


"Big bulk processing, like graphics or matrices"

The Reality - Compute Node



The Reality - Compute Node



How Does CUDA Fit In?

- CUDA is C++ with some additional propriety API.
- CUDA allows for surprisingly clean SIMD programming.
- CPU SIMD programming is hard, if the compiler doesn't do it right, then intrinsics are usually the only option.
- CUDA SIMD programming makes everything SIMD by default.
- Dominant platform for GPU computing in scientific research, AI/ML, and high-performance computing.
- CUDA only works for Nvidia GPUs.

CUDA Architecture Concepts

- **Host:** The CPU and its system memory. Programs start in host code.
- **Device:** The GPU and its dedicated memory. This is where CUDA kernels execute.
- **Kernels:**
  - Functions written in CUDA C/C++ that execute on the GPU.
  - Invoked from the host code.
  - Executed by many threads in parallel.
- **Execution Components:**
  - Kernels aren't executed on the entire GPU. Instead, they are assigned parts of the GPU.
  - **Thread:** Not like CPU threads.
    - Think CPU SIMD on a 16 unit vector. A CUDA thread is 1 unit of that 16 unit vector.
  - **Warp:** A group of CUDA threads (has so far always been 32 threads).
    - Useful for understanding scheduling and branching.
  - **Block:** A user defined group of threads.
  - **Grid:** How many blocks.
- How this all maps to architecture is important and will be covered later.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Intro Code Example

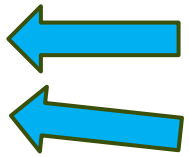
```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Compile with:

/usr/local/cuda-12/bin/nvcc -arch=sm_86 vector.cu -o vector.x

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```



Store device addresses. The host can't access these directly. But the host tracks them.

cudaMalloc is how to allocate GPU memory. (CUDA kernels can't allocate their own memory, only host code can.)

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Copy host data into the GPU.
This is usually slow (relatively).
Too many host-to-device and device-to-host copies means this will be your bottleneck

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Compute how many blocks of threads are needed.
A good block size is usually between 128 and 256.
Avoid going over 256 threads per block (this will use up too many registers.)
These can be done in a 2D and 3D scheme. My advice: never do that. 1D keeps things straightforward.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Start running the GPU function!
The <<< >>> is CUDA specific syntax.
The number of blocks and threads per block specified.
Host arguments are **copied** into the GPU.
The kernel will launch and run.
The CPU/host thread is not blocked, so it will just keep running. (If a second kernel is invokved in this fashion right afterward, then they are sequenced, kernels are invoked one after the other, then those kernels are sequenced, the second cannot start until the first ends.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

The join() of CUDA. Host thread waits for all queued up GPU actions to complete.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

The join() of CUDA. Host thread waits for all queued up GPU actions to complete.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

A strange part of CUDA code. Every programmer almost always needs to compute a unique thread ID.

The uniqueness allows this thread to work on unique regions, thus avoiding race conditions.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

The work!
This is SIMD!
Despite looking like normal C++, CUDA and the GPU ensures one instruction is sent to groups of threads.
The SIMD is usually wider than CPU AVX-512.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

A branch.

If all threads go into the if statement, SIMD continues.

If some threads go into the if statement, the SIMD threads are split into groups.

Here one group of threads runs their code and the others are essentially masked out.

Had there been an else, then a group of thread are split in two. One group of threads runs one SIMD instruction in the if, another SIMD instruction runs the other threads in the else.

(Avoid heavy branching per thread, it can turn SIMD back into regular SISD, and then the GPU is worse than CPUs.)

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Check for errors. This is your only chance. A crashing kernel just returns unless you check error codes.

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

Get the computed data out of GPU memory.
This is also usually slow (relatively)

Intro Code Example

```
// Device code (Kernel definition)
__global__ void myKernel(float* device_input, float* device_output, int N) {
    // Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        device_output[idx] = device_input[idx] * 2.0f; // Example operation
    }
}

// Host code
int main() {
    // ... (Host memory allocation and initialization) ...

    // Device memory allocation
    float *d_input, *d_output;
    cudaMalloc((void**)&d_input, dataSize);
    cudaMalloc((void**)&d_output, dataSize);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, dataSize, cudaMemcpyHostToDevice);

    // Define grid and block dimensions
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    // Launch the kernel
    myKernel<<<numBlocks, blockSize>>>(d_input, d_output, N);

    cudaDeviceSynchronize(); // Wait for kernel to complete
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        printf("CUDA error: %s\n", cudaGetErrorString(err));
    }

    // Copy results from device to host
    cudaMemcpy(h_output, d_output, dataSize, cudaMemcpyDeviceToHost);

    // ... (Host memory deallocation) ...
    return 0;
}
```

While this program computes correctly,
It's actually a bad program!

Due to device <-> host transfer costs being worse than memory bandwidth costs, this should be computed on the CPU.

General Rules of Thumb

- Very short or fast kernels usually lose to data copy in/copy out times.
- The Nvidia GPU scheduler is really, really good. Clean problems run on 100s+ blocks. The scheduler will then schedule those blocks to any open part of the GPU.
- Most SIMD problems port well to GPUs.
- Most non-SIMD problems get worse times on GPUs compared to CPUs.
- Keep CUDA threads running together. Don’t let them diverge. Diverging problems should just use CPUs.

General Rules of Thumb (Continued)

- GPU RAM tends to have much greater throughput than CPU RAM.
- Memory access patterns are critical, even more than CPUs. GPUs have “cache lines”, but they call it coalescing. The cache lines are typically larger than CPU cache lines. Ensure threads access data in these same coalescing regions.
- Extremely few problems are compute bound on GPUs. So much so that it usually works to estimate that the computations are infinitely fast, and instead it’s all memory access patterns and times.
- Very few problems make sense to extract full machine potential by running the algorithm on both CPUs and GPUs simultaneously.
- Complicated problems will do some parts on CPUs, and then other parts on GPUs.

Nvidia GPU Gotchas

- CPU libraries don't work in CUDA code. No STL!
- Awkward random numbers.
- printf() is one of the few tools that work also on the GPU.
  - But it doesn't print directly to console. It instead writes to a buffer, and then on successful kernel execution the buffer is copied out, and then it is printed.
  - A crashing kernel won't transmit the buffer.
  - The buffer can fill up, so heavy logging runs out of room.
- Breakpoint/stepping debug tools exist for GPU CUDA code, but they take effort to implement.
- Nvidia GPUs have thousands of registers, but each thread gets its own set of registers. That means registers run out quickly. Don't go above 256 threads per block.
- No dynamic allocation.
- No easy way to detect where codes crashed.
- Fast interconnectivity exists with NVLink. If you have the money for it.
- Nvidia's C++ compiler is buggy for complex templating.
- Many of the CPUs tricks don't work as well or at all on GPUs, such as branch prediction or out-of-order execution.
- Best code is Fortran-like! Clever C++ code is usually a bad idea.

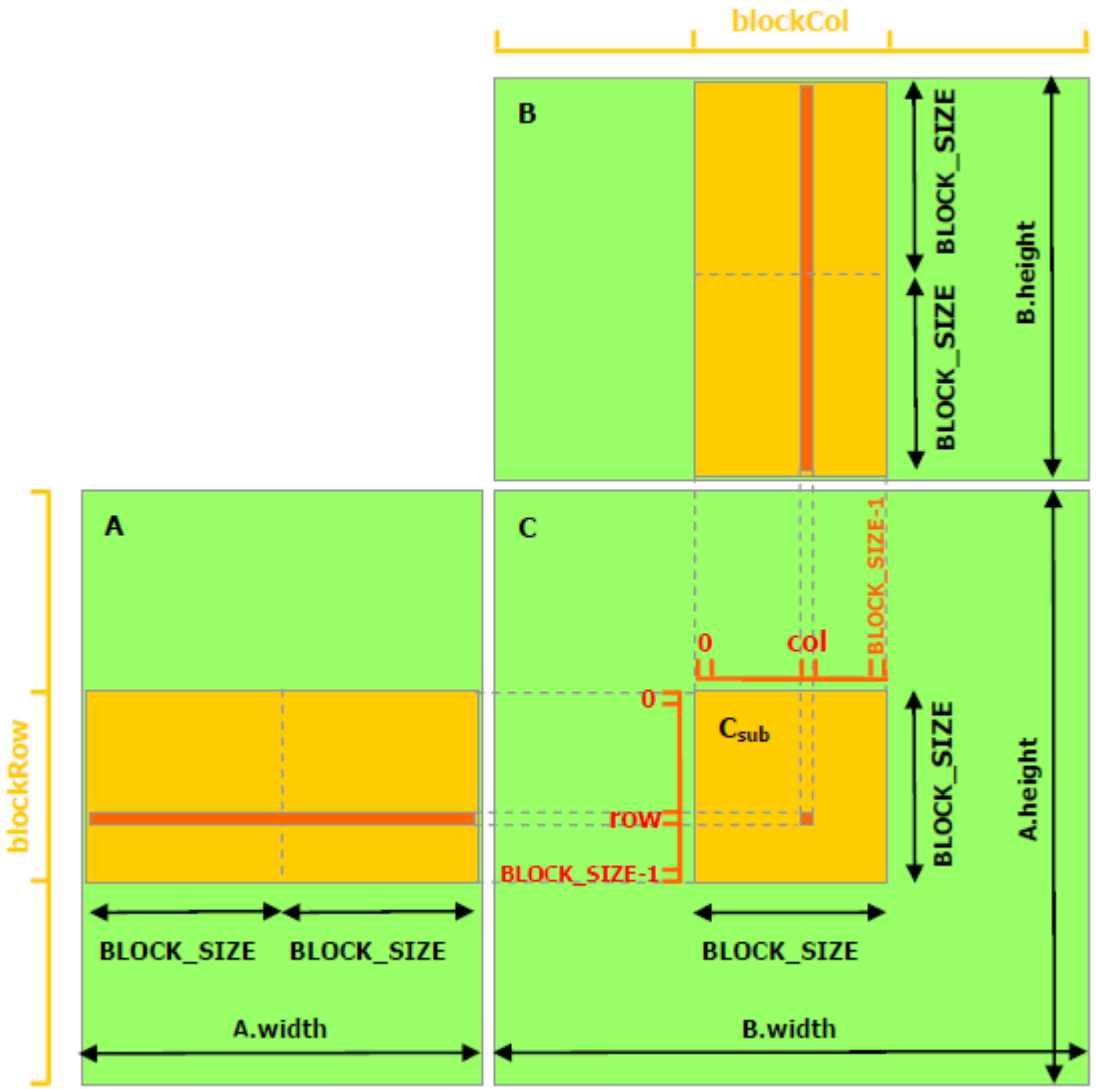
A Few Intro Items Remain

- Blocks have some coordination mechanisms
  - __syncthreads() is a barrier, all threads must arrive before any can continue.
  - __syncwarp() is a similar barrier but syncs only within a warp (finer grained).
- Shared memory
  - Allows GPU threads in a block to read and write each other's data without requiring the sharing exist all the way out in RAM.
  - Shared memory performance is typically on the order L1 cache.
- Constant memory
  - Not discussed as often. But should be.
  - These are essentially memory that all threads can see.
  - Performance is as fast as registers!
  - Best when registers get used up.
- Texture memory
  - Best for 2D memory layouts (helps avoid coalescing issues).

CPU Hardware Core ≈ GPU Streaming Multiprocessor

- A Streaming Multiprocessor is frequently termed an SM.
- Nvidia GPUs have up to dozens of these SMs.
- Latest generations have subdivided SMs further, *roughly* like CPU hyperthreading, but more explicit.
- A V100 SM is shown on the right.
- Each tiny square is like an individual component of a CPU SIMD pipeline.

Example #2 – Matrix-Matrix Multiply



- The goal is to reuse data by first loading matrix data in RAM into shared memory (orange blocks).
- Once in shared memory, values can be read multiple times.
- The idea is that many shared memory reads is much, much faster than many RAM reads.
- Using shared memory almost always means race conditions unless barriers are used.
- C does not use shared memory. The solution elements in C are computed in parts and all parts accumulated together.
- This algorithm is not at all obvious at first glance. Feel free to work it out on paper.

Example #2 – Matrix-Matrix Multiply

```
// Matrix multiplication - Host code
// Matrix dimensions are assumed to be multiples of BLOCK_SIZE
void MatMul(const Matrix A, const Matrix B, Matrix C)
{
    // Load A and B to device memory
    Matrix d_A;
    d_A.width = d_A.stride = A.width; d_A.height = A.height;
    size_t size = A.width * A.height * sizeof(float);
    cudaMalloc(&d_A.elements, size);
    cudaMemcpy(d_A.elements, A.elements, size, cudaMemcpyHostToDevice);
    Matrix d_B;
    d_B.width = d_B.stride = B.width; d_B.height = B.height;
    size = B.width * B.height * sizeof(float);
    cudaMalloc(&d_B.elements, size);
    cudaMemcpy(d_B.elements, B.elements, size, cudaMemcpyHostToDevice);
    // Allocate C in device memory
    Matrix d_C;
    d_C.width = d_C.stride = C.width; d_C.height = C.height;
    size = C.width * C.height * sizeof(float);
    cudaMalloc(&d_C.elements, size);
    // Invoke kernel
    dim3 dimBlock(BLOCK_SIZE, BLOCK_SIZE);
    dim3 dimGrid(B.width / dimBlock.x, A.height / dimBlock.y);
    MatMulKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C);
    // Read C from device memory
    cudaMemcpy(C.elements, d_C.elements, size,
               cudaMemcpyDeviceToHost);
    // Free device memory
    cudaFree(d_A.elements);
    cudaFree(d_B.elements);
    cudaFree(d_C.elements);
}
```

- The code on the left is just host code driver
  - Typical process of preparing host and device memory
  - Invoking the kernel
- The classic example from CUDA documentation
  - Better than naïve, but better implementations exist
  - New hardware has matrix-matrix multipliers built in
  - Almost always use existing libraries now
  - This example given to show both shared memory and __syncthreads

Example #2 – Matrix-Matrix Multiply

```
// Matrices are stored in row-major order:
// M(row, col) = *(M.elements + row * M.stride + col)
typedef struct {
    int width;
    int height;
    int stride;
    float* elements;
} Matrix;

// Get a matrix element
__device__ float GetElement(const Matrix A, int row, int col)
{
    return A.elements[row * A.stride + col];
}

// Set a matrix element
__device__ void SetElement(Matrix A, int row, int col, float value)
{
    A.elements[row * A.stride + col] = value;
}

// Get the BLOCK_SIZExBLOCK_SIZE sub-matrix Asub of A that is
// located col sub-matrices to the right and row sub-matrices down
// from the upper-left corner of A
 __device__ Matrix GetSubMatrix(Matrix A, int row, int col)
{
    Matrix Asub;
    Asub.width    = BLOCK_SIZE;
    Asub.height   = BLOCK_SIZE;
    Asub.stride   = A.stride;
    Asub.elements = &A.elements[A.stride * BLOCK_SIZE * row
                                         + BLOCK_SIZE * col];
    return Asub;
}
```

- Supporting code
- The struct exists for both CPU and GPU codes
- Three device functions
  - Get a row/col value
  - Set a row/col value
  - Return a sub matrix (the stride helps make this work)

Example #2 – Matrix-Matrix Multiply

```
// Matrix multiplication kernel called by MatMul()
 __global__ void MatMulKernel(Matrix A, Matrix B, Matrix C)
{
    // Block row and column
    int blockRow = blockIdx.y;
    int blockCol = blockIdx.x;
    // Each thread block computes one sub-matrix Csub of C
    Matrix Csub = GetSubMatrix(C, blockRow, blockCol);
    // Each thread computes one element of Csub
    float Cvalue = 0;
    // Thread row and column within Csub
    int row = threadIdx.y;
    int col = threadIdx.x;
    // Loop over all the sub-matrices of A and B
    // Multiply each pair of sub-matrices together and accumulate results
    for (int m = 0; m < (A.width / BLOCK_SIZE); ++m) {
        // Get sub-matrix Asub of A
        Matrix Asub = GetSubMatrix(A, blockRow, m);
        // Get sub-matrix Bsub of B
        Matrix Bsub = GetSubMatrix(B, m, blockCol);
        // Shared memory used to store Asub and Bsub respectively
        __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
        __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];
        // All threads cooperate to load 16x16 region
        As[row][col] = GetElement(Asub, row, col);
        Bs[row][col] = GetElement(Bsub, row, col);
        // Synchronize, all threads had to cooperate.
        __syncthreads();
        // Multiply Asub and Bsub together
        for (int e = 0; e < BLOCK_SIZE; ++e)
            Cvalue += As[row][e] * Bs[e][col];
        // Synchronize again (otherwise we get one thread that
        // loops around again and reads into shared memory)
        __syncthreads();
    }
    // Write Csub to device memory
    // Each thread writes one element
    SetElement(Csub, row, col, Cvalue);
}
```

- Supporting code
- The struct exists for both CPU and GPU codes
- Three device functions
  - Get a row/col value
  - Set a row/col value
  - Return a sub matrix (the stride helps make this work)

Example #3 – Odd-Even Sort

```
__global__ void oddEvenSort(int *a, int n)
{
  __shared__ isSorted;
  int i = blockIdx.x * blockDim.x + threadIdx.x; // Unique thread ID

  do {
    isSorted = 1;
    __syncthreads(); // Assume its all sorted until proven otherwise

    if ( i % 2 == 0 && i < n-1 ) {
       // even phase
       if ( a[i] > a[i+1] ) {
          swap( a[i], a[i+1] );
          isSorted = 0; // Something swapped, do another while loop iteration
       }
    }
    __syncthreads(); // All threads must finish before move on

    if ( i % 2 == 1 && i < n-1 ) {
       // odd phase
       if ( a[i] > a[i+1] ) {
          swap( a[i], a[i+1] );
          isSorted = 0; // Something swapped, do another while loop iteration
       }
    }
    __syncthreads(); // Wait for all threads to finish their work.
  } while ( isSorted == 0 );

}
```

- Just CUDA code shown.
- Many race conditions averted by ___syncthreads() use.
- No race condition inside if statement, multiple writes of 0 simultaneously isn't a problem.
- Half of all threads aren't used! This is rather inefficient.
- Much like the STL, often basic things have already been completed, and using libraries is superior.
- Examples here are for CUDA introductions to code bigger and uglier things.

Conclusions

- Nvidia's CUDA does a wonderful job creating a natural design pattern for SIMD programming on Nvidia GPUs.
- Code in mostly native C or C++. The compiler will run SIMD on the rest.
- Threads in CUDA aren't like threads in CPUs.
  - Each CUDA thread is more like a SIMD component.
  - Each CUDA thread does not get its own call stack.
  - But each CUDA thread does get its own registers and variables.
- Break a problem down into groups of threads (each group is a block).
- Good programs create many, many blocks. Then the GPU scheduler runs those blocks on open SMs.
- Ensure data is copied into and out of the GPU as needed.
  - This here is often too much of a performance bottleneck.
- Register reuse, cache lines, and locality are huge factors.
  - GPUs work well on these problems.
  - Otherwise, CPUs are usually best.